15-418 Project Proposal: CUDA Optimization of Multi-Head Attention

Dominik Djajadi (ddjajadi)

Fall 2026

Project website: `https://DominikDjajadi.github.io/cuda-mha-15418-site/`

1 Summary

I aim to implement and optimize a complete multi-head attention (MHA) forward pass using CUDA C++. I plan to investigate parallelism both across and within attention heads, with an emphasis on communication, memory traffic, and profiling-guided optimizations. I will check correctness against PyTorch and compare a straightforward CUDA baseline, intermediate optimizations, and the final implementation.

2 Background

Multi-head attention is used in Transformers to let tokens incorporate information from other tokens. Learned weight matrices project the input into queries, keys, and values. Different heads use different projections, allowing the model to learn different attention patterns.

For head h, with head dimension d_h, the computation is

$$Q_h = XW_Q^{(h)}, \qquad K_h = XW_K^{(h)}, \qquad V_h = XW_V^{(h)},$$
$$O_h = \mathrm{softmax}\left(\frac{Q_h K_h^{\mathsf{T}}}{\sqrt{d_h}}\right) V_h.$$

The outputs of the heads are concatenated and passed through an output projection. I plan to implement this complete forward layer, including the Q/K/V and output projections, initially using unmasked attention.

For a sequence of length N, the attention score matrix for one head has N^2 entries, and computing it requires $\Theta(N^2 d_h)$ work. Doubling N therefore roughly quadruples both the attention work and the score-matrix size when the other dimensions are fixed. Large intermediates can cause considerable memory traffic and may not fit in GPU memory for long sequences.

3 The Challenge

Although much of MHA is parallelizable, achieving good GPU performance is not automatic. The main challenges I want to investigate are:

- **Communication and synchronization:** Softmax needs row-wise maximum and sum reductions across threads; warp and block layouts affect synchronization costs.
- **Memory traffic and locality:** Separate kernels may repeatedly write and read large intermediates. Coalesced accesses and shared-memory reuse can reduce traffic.
- **Work distribution:** One block per head may underutilize the GPU; splitting heads across blocks exposes more parallelism but may repeat K/V loads.
- **Resource tradeoffs:** Larger tiles improve reuse but may reduce occupancy through register and shared-memory demand.

I plan to profile the baseline first, apply optimizations to measured bottlenecks, and remeasure. Improvements to one component may expose another as the new bottleneck, so the end-to-end layer performance matters more than any individual kernel's speedup.

4 Resources and Platform Choice

I will write native CUDA C++ kernels from scratch and benchmark primarily on GHC GPUs. Synthetic FP32 inputs and weights will be validated against PyTorch, also used as a performance reference. CUDA events and NVIDIA profiling tools will guide optimization. I may compare against my local RTX 4050 GPU if time permits. The CUDA documentation and FlashAttention papers will serve as references, not source code.

CUDA fits this problem because MHA has independent heads, query-level parallelism, matrix operations, and reductions sensitive to GPU memory and thread organization.

5 Goals and Deliverables

Minimum: A correct FP32 implementation of the full MHA forward pass, including custom CUDA matrix multiplication and softmax kernels, validated against PyTorch.

Target: Two or three substantial optimizations chosen through profiling. Possible directions include improved reductions, memory layouts and reuse, work partitioning across and within heads, and a limited form of kernel fusion. I will retain intermediate versions to show which changes helped and which did not.

Evaluation: Measure end-to-end and component runtimes across representative batch sizes, sequence lengths, head counts, and head dimensions. Compare fixed-workload speedup, latency-budget throughput, and memory capacity. These connect to the three scaling perspectives from class, although one GPU cannot establish formal resource scaling. I do not assume I can outperform PyTorch.

Reach goals: More extensive kernel fusion or IO-aware attention with online softmax, causal masking, mixed precision/Tensor Cores, or comparison across GPUs.

Fallback: If full-layer optimization takes longer than expected, focus the deeper performance investigation on scaled dot-product attention while retaining a correct full-layer baseline if feasible.

6 Schedule

Dates	Planned work
Nov 9–15	Review the algorithm and CUDA environment; begin the baseline. Limited availability this week.
Nov 16–22	Complete and validate the forward baseline; begin initial optimizations.
Nov 23	Milestone: working baseline, initial timing, and an updated optimization plan.
Nov 24–29	Profile bottlenecks and optimize important components.
Nov 30–Dec 6	Investigate communication, memory traffic, work partitioning, and selected fusion.
Dec 7–10	Finish comparisons, interpret performance, and prepare final deliverables.

References

[1] T. Dao et al., *FlashAttention* (2022), `https://arxiv.org/abs/2205.14135`. [2] T. Dao, *FlashAttention-2* (2023), `https://arxiv.org/abs/2307.08691`. [3] NVIDIA, *CUDA C++ Programming Guide*, `https://docs.nvidia.com/cuda/cuda-c-programming-guide/`.

AI Assistance

AI was used to help format my proposal in LaTeX and refine wording based on my project plans and explanations.